

Unidad Didáctica

**Elementos Básicos del Lenguaje de
Programación, Unidad II**

Contenido

Introducción	3
Desarrollo de Contenidos	4
Lenguaje de Programación	4
Orígenes del Lenguaje de programación	4
Características del Lenguaje	5
Estructura de un programa.....	6
Compilación y Depuración	7
Elementos de un Programa.....	8
Tipos de Operadores.....	8
Tipos de Datos	9
Variables, constantes y expresiones	9
Expresiones de Entrada/Salida.....	10
Estructuras de Control	11
Edición, compilación y enlace de un programa sencillo en un entorno de programación	15
Bibliografía y Webgrafía	16
Glosario	17

Introducción



¡Bienvenido al fascinante mundo de la programación! En este tema, nos sumergiremos en el apasionante mundo del lenguaje de programación. Desde sus orígenes hasta las características más destacadas, pasando por la estructura de un programa, tipos de datos, operadores y

estructuras de control, entre otros aspectos esenciales.

El lenguaje de programación ha tenido una evolución interesante desde sus inicios, y conocer su historia es fundamental para comprender su funcionamiento actual. Conocer los orígenes, sus creadores y la evolución del lenguaje es un paso importante para comprender la importancia de esta herramienta en la actualidad.

Una vez comprendidos los orígenes, exploraremos las características del lenguaje, desde su sintaxis y semántica hasta su capacidad para la resolución de problemas y la automatización de tareas. También exploraremos la estructura de un programa, los elementos que lo componen y cómo se relacionan entre sí para crear una solución lógica.

Además, analizaremos la compilación y depuración de programas, elementos esenciales para garantizar su correcto funcionamiento. También exploraremos los tipos de operadores, datos, variables, constantes y expresiones, y cómo estos elementos pueden ser combinados para crear programas útiles y eficientes.

Por último, nos adentraremos en las estructuras de control, que nos permiten crear programas que se ajusten a situaciones específicas, y conoceremos el proceso

completo de edición, compilación y enlace de un programa sencillo en un entorno de programación. ¡Prepárate para descubrir el apasionante mundo de la programación!

Desarrollo de Contenidos

Lenguaje de Programación

Orígenes del Lenguaje de programación

El lenguaje de programación es un conjunto de instrucciones que se utilizan para comunicarse con un ordenador y decirle qué hacer. Los orígenes del lenguaje de programación se remontan a la década de 1840, cuando el matemático británico Ada Lovelace escribió lo que se considera el primer algoritmo destinado a ser procesado por una máquina. Sin embargo, el verdadero comienzo del lenguaje de programación moderno se sitúa en la década de 1950, cuando los programadores comenzaron a trabajar en los primeros lenguajes de programación de alto nivel.

El primer lenguaje de programación de alto nivel se llama Fortran (acrónimo de FORMula TRANslation) y fue desarrollado en 1957 por IBM para simplificar la programación de cálculos científicos complejos. También en la década de 1950, el matemático John McCarthy desarrolló Lisp, un lenguaje de programación utilizado principalmente en la investigación y el desarrollo de la inteligencia artificial.

En la década de 1960, el lenguaje de programación COBOL (acrónimo de COMmon Business Oriented Language) fue desarrollado para facilitar la programación de aplicaciones empresariales y financieras. También en la década de 1960, se desarrolló el lenguaje de programación BASIC (Beginner's All-purpose Symbolic Instruction Code), diseñado para ser fácil de aprender y utilizado en la educación.

En la década de 1970, surgieron lenguajes de programación orientados a objetos como Simula y Smalltalk, que permitían a los programadores construir programas

con objetos que representaban entidades del mundo real. También en la década de 1970, se desarrolló el lenguaje de programación C, que se convirtió en uno de los lenguajes de programación más populares de la historia debido a su eficiencia y portabilidad.

Desde entonces, han surgido muchos otros lenguajes de programación, cada uno con sus propias características y usos. Los lenguajes de programación más populares hoy en día incluyen Java, Python, C++, JavaScript y PHP.

Características del Lenguaje

Algunas características del lenguaje de programación Python son:

- Fácil de aprender: Python tiene una sintaxis clara y legible que lo hace fácil de entender y aprender, especialmente para aquellos que son nuevos en la programación.
- Interpretado: Python es un lenguaje de programación interpretado, lo que significa que no necesita ser compilado antes de ejecutarse. Esto hace que la programación en Python sea más rápida y eficiente.
- Multiplataforma: Python se puede ejecutar en diferentes plataformas, como Windows, Linux, macOS, entre otros.
- Orientado a objetos: Python es un lenguaje de programación orientado a objetos, lo que significa que se basa en la idea de objetos y clases. Esto facilita la reutilización de código y hace que el código sea más modular y escalable.
- Tipado dinámico: Python es un lenguaje de programación de tipado dinámico, lo que significa que el tipo de una variable se infiere en tiempo de ejecución. Esto hace que la programación en Python sea más flexible y fácil de escribir.

- Amplia biblioteca estándar: Python viene con una biblioteca estándar integrada que proporciona una amplia gama de módulos y funciones para realizar tareas comunes, lo que facilita la programación.
- Alto nivel: Python es un lenguaje de programación de alto nivel, lo que significa que tiene una sintaxis y una estructura de datos más intuitivas, lo que facilita la programación.
- Gran comunidad: Python tiene una gran comunidad de desarrolladores que comparten su conocimiento y recursos, lo que hace que sea fácil encontrar respuestas a preguntas y soluciones a problemas comunes.

Estructura de un programa

La estructura de un programa en Python se divide en dos partes principales: la definición de funciones y el código principal.

La definición de funciones es la primera parte del programa, y se utiliza para definir las funciones que se utilizarán en el código principal. Las funciones son bloques de código que realizan una tarea específica y se pueden llamar desde otras partes del programa. La sintaxis para definir una función en Python es:

```
def nombre_de_la_funcion(parametros):  
    # Cuerpo de la función  
    return resultado
```

El código principal es la segunda parte del programa y contiene la secuencia de instrucciones que se ejecutan una vez que se han definido las funciones. El código principal puede llamar a las funciones definidas previamente y realizar cualquier otra operación necesaria. La sintaxis general para el código principal es simplemente una secuencia de instrucciones escritas en orden.

Una estructura típica de un programa en Python se vería así:

```

# Definición de funciones
def funcion_1(parametros):
    # Cuerpo de la función
    return resultado

def funcion_2(parametros):
    # Cuerpo de la función
    return resultado

# Código principal
# Llamada a la función 1
resultado_1 = funcion_1(parametros_1)

# Llamada a la función 2
resultado_2 = funcion_2(parametros_2)

# Otras instrucciones

```

Compilación y Depuración

La compilación y la depuración son dos procesos importantes en la programación de Python.

La compilación se refiere al proceso de convertir el código fuente escrito en Python a un archivo ejecutable, que la computadora pueda comprender y ejecutar. A diferencia de otros lenguajes de programación, Python es un lenguaje interpretado, lo que significa que no se compila en un archivo ejecutable antes de su ejecución. En su lugar, el intérprete de Python lee y ejecuta el código fuente directamente en tiempo de ejecución. Sin embargo, aún se pueden crear archivos ejecutables utilizando herramientas como py2exe o PyInstaller. Estas herramientas empaquetan el código fuente de Python y los archivos necesarios en un archivo ejecutable que se puede ejecutar en una computadora sin la necesidad de tener Python instalado.

La depuración se refiere al proceso de identificar y corregir errores o errores en el código fuente. Los errores son inevitables en la programación, pero la depuración es una parte importante del proceso de desarrollo de software. En Python, existen varias herramientas de depuración disponibles, como el depurador de Python (PDB), PyCharm Debugger y Visual Studio Code Debugger. Estas herramientas le permiten

establecer puntos de interrupción en el código, examinar variables y ejecutar el código paso a paso para identificar y corregir errores.

En resumen, la compilación y la depuración son procesos importantes en la programación de Python. La compilación se refiere a la conversión del código fuente de Python a un archivo ejecutable, mientras que la depuración se refiere al proceso de identificar y corregir errores en el código fuente. Ambos procesos son esenciales para el desarrollo de software exitoso en Python.

Elementos de un Programa

Tipos de Operadores

- Operadores aritméticos: se utilizan para realizar operaciones matemáticas, como suma (+), resta (-), multiplicación (*), división (/), módulo (%), exponenciación (**), división entera (/).
- Operadores de asignación: se utilizan para asignar valores a variables, como el operador de asignación (=), que asigna un valor a una variable.
- Operadores de comparación: se utilizan para comparar dos valores, como igualdad (==), desigualdad (!=), mayor que (>), menor que (<), mayor o igual que (>=), menor o igual que (<=).
- Operadores lógicos: se utilizan para combinar valores lógicos y producir un resultado lógico, como el operador AND (and), OR (or) y NOT (not).
- Operadores de identidad: se utilizan para comparar la identidad de dos objetos, como el operador is, que devuelve True si dos objetos son el mismo objeto en memoria.
- Operadores de pertenencia: se utilizan para comprobar si un valor está presente en una secuencia, como el operador in, que devuelve True si un valor está presente en una secuencia.

- Operadores bit a bit: se utilizan para realizar operaciones bit a bit en valores enteros, como el operador AND bit a bit (&), OR bit a bit (|), XOR bit a bit (^), desplazamiento a la izquierda (<<) y desplazamiento a la derecha (>>).

Tipos de Datos

- Números: incluyen números enteros, números de punto flotante y números complejos.
- Cadena de caracteres (strings): secuencia de caracteres alfanuméricos y especiales, se delimitan con comillas simples o dobles.
- Booleanos: son valores lógicos True (verdadero) y False (falso), utilizados para expresar la veracidad o falsedad de una condición.
- Listas: son una colección ordenada y mutable de elementos, se delimitan por corchetes y los elementos se separan por comas.
- Tuplas: son similares a las listas, pero son inmutables, se delimitan por paréntesis y los elementos se separan por comas.
- Conjuntos (sets): son una colección no ordenada y mutable de elementos únicos, se delimitan por llaves y los elementos se separan por comas.
- Diccionarios: son una colección mutable de pares clave-valor, se delimitan por llaves y los elementos se separan por comas. Cada elemento consta de una clave y un valor separados por dos puntos.

Variables, constantes y expresiones

En Python, una variable es un espacio reservado en la memoria para almacenar un valor. Las variables se crean en Python simplemente asignando un valor a un nombre, y pueden ser modificadas durante la ejecución del programa. Por ejemplo:

```
x = 5  
y = 2  
z = x + y
```

En este ejemplo, se crean las variables `x` e `y` y se les asignan los valores 5 y 2 respectivamente. Luego, se crea la variable `z` y se le asigna el resultado de la expresión `x + y`, que es 7.

Una constante, por otro lado, es un valor que no cambia durante la ejecución del programa. En Python, las constantes se suelen definir como variables globales, cuyo valor se asigna al principio del programa y no se modifica a lo largo del mismo. Por convención, los nombres de las constantes se escriben en mayúsculas para distinguirlas de las variables. Por ejemplo:

```
PI = 3.14159  
GRAVEDAD = 9.8
```

Una expresión es una combinación de valores, variables y operadores que Python puede evaluar para producir un resultado. Por ejemplo, la expresión `3 + 5` evalúa a 8, la expresión `x * y` evalúa al producto de los valores de `x` e `y`, y la expresión `z > 10` evalúa a `True` o `False` dependiendo del valor de `z`. Las expresiones se utilizan en Python para realizar cálculos y tomar decisiones basadas en el valor de ciertas variables o condiciones.

Expresiones de Entrada/Salida

En Python, las expresiones de entrada y salida se utilizan para interactuar con el usuario y mostrar información en la consola.

Para la entrada de datos, se utiliza la función `input()`, que permite al usuario introducir datos a través del teclado. Esta función devuelve una cadena de caracteres que puede ser almacenada en una variable.

Ejemplo:

```
nombre = input("Por favor, introduce tu nombre: ")  
print("Hola " + nombre + "!")
```

En este ejemplo, la función `input()` solicita al usuario que introduzca su nombre, y la cadena resultante se almacena en la variable `nombre`. Luego, se utiliza la función `print()` para mostrar un mensaje que incluye el nombre del usuario.

Para la salida de datos, se utiliza la función `print()`, que permite mostrar información en la consola. Esta función puede mostrar cadenas de caracteres y valores numéricos, y también permite formatear la salida de datos.

Ejemplo:

```
precio = 25.99  
cantidad = 3  
total = precio * cantidad  
print("El precio unitario es: $" + str(precio))  
print("La cantidad es: " + str(cantidad))  
print("El total es: $" + str(total))
```

En este ejemplo, se utilizan las variables `precio` y `cantidad` para calcular el total de la venta, y luego se muestra la información en la consola utilizando la función `print()`. Las funciones `str()` se utilizan para convertir los valores numéricos en cadenas de caracteres antes de mostrarlos en la consola.

Estructuras de Control

Secuenciación

En Python, la estructura de control de secuenciación se utiliza para ejecutar una serie de instrucciones en orden secuencial, una detrás de la otra. Esto se logra a través del uso de sentencias de asignación y expresiones de llamada a funciones.

Las sentencias de asignación permiten asignar valores a variables y se utilizan para almacenar datos que se utilizarán en el futuro en el programa. Por ejemplo, en la secuencia "x = 2 + 3", la variable x se asigna con el valor 5.

Las expresiones de llamada a funciones se utilizan para llamar a funciones predefinidas o definidas por el usuario. Estas funciones pueden realizar cálculos o tareas específicas en el programa. Por ejemplo, en la secuencia "print('Hola mundo')", la función print() se utiliza para mostrar el mensaje "Hola mundo" en la consola.

La estructura de control de secuenciación en Python se implementa utilizando bloques de código, que se delimitan mediante indentación en lugar de llaves o palabras clave. Esto significa que todas las instrucciones que pertenecen a un bloque de código deben tener la misma cantidad de espacios en blanco antes de la instrucción. Por ejemplo:

```
if x > 0:  
    print("x es positivo")  
    y = x * 2  
    print("El doble de x es", y)
```

En este ejemplo, las tres instrucciones están en el mismo bloque de código y se ejecutan secuencialmente si la condición "x > 0" es verdadera. La indentación indica que estas instrucciones están en el mismo bloque de código. Si la condición es falsa, las instrucciones dentro del bloque no se ejecutarán.

Selección

La estructura de control Selección en Python permite que un programa ejecute diferentes instrucciones de acuerdo a una condición específica. En otras palabras, se puede establecer una condición y si ésta se cumple, se ejecuta un bloque de

código, y si no se cumple, se ejecuta otro bloque de código. La estructura de control Selección en Python se puede implementar mediante las siguientes estructuras:

Declaraciones if: Esta declaración se utiliza para ejecutar un bloque de código si se cumple una condición determinada. Si la condición no se cumple, el bloque de código no se ejecuta. La sintaxis de esta estructura es la siguiente:

```
if condición:  
    bloque de código
```

Declaraciones if-else: Esta declaración se utiliza para ejecutar un bloque de código si se cumple una condición determinada y otro bloque de código si no se cumple. La sintaxis de esta estructura es la siguiente:

```
if condición:  
    bloque de código 1  
else:  
    bloque de código 2
```

Declaraciones if-elif-else: Esta declaración se utiliza cuando se tienen múltiples condiciones que deben cumplirse y se ejecuta un bloque de código específico para cada una de las condiciones. La sintaxis de esta estructura es la siguiente:

```
if condición 1:  
    bloque de código 1  
elif condición 2:  
    bloque de código 2  
else:  
    bloque de código 3
```

En general, la estructura de control Selección en Python permite que un programa tome decisiones basadas en las condiciones establecidas y ejecute diferentes acciones según sea el caso.

Iteración

La estructura de control Iteración en Python es utilizada para repetir una o varias acciones de manera iterativa, mientras se cumpla una condición. En Python, existen dos tipos de estructuras de control de iteración: el bucle "while" y el bucle "for".

Bucle while: El bucle "while" es una estructura de control de iteración que repite un bloque de código mientras se cumple una condición dada. La estructura de sintaxis del bucle while es la siguiente:

```
while condición:  
    # Cuerpo del bucle
```

El bloque de código dentro del bucle se ejecutará mientras se cumpla la condición. La condición debe ser evaluada como verdadera o falsa en cada iteración del bucle. Si la condición es falsa, el bucle terminará.

Bucle for: El bucle "for" es una estructura de control de iteración que permite iterar sobre un conjunto de elementos. La estructura de sintaxis del bucle for es la siguiente:

```
for variable in secuencia:  
    # Cuerpo del bucle
```

En cada iteración del bucle, la variable toma el valor de uno de los elementos de la secuencia, y se ejecuta el bloque de código dentro del bucle. La secuencia puede ser una lista, una tupla, un conjunto, un diccionario o cualquier otro tipo de objeto iterable.

En resumen, la estructura de control de iteración en Python se utiliza para ejecutar un bloque de código repetidamente mientras se cumpla una condición o para iterar sobre un conjunto de elementos.

Edición, compilación y enlace de un programa sencillo en un entorno de programación

Para editar, compilar y enlazar un programa sencillo en Python, se pueden seguir los siguientes pasos:

1. Abrir un editor de texto: Se puede usar cualquier editor de texto como Notepad, Sublime Text, Atom, Visual Studio Code, entre otros.
2. Escribir el código del programa: Se puede escribir el código del programa en el editor de texto. Por ejemplo, se puede crear un archivo llamado "ejemplo.py" y escribir el siguiente código:

```
# Este es un programa sencillo en Python que imprime un  
mensaje en la consola  
  
print("Hola, mundo!")
```

3. Guardar el archivo: Después de escribir el código del programa, se debe guardar el archivo con una extensión .py (por ejemplo, ejemplo.py).
4. Compilar el programa: En Python, no es necesario compilar el programa de forma explícita, ya que Python es un lenguaje interpretado. Sin embargo, se puede verificar la sintaxis del programa usando el comando "python -m py_compile archivo.py" en la terminal. En este caso, se usaría el comando "python -m py_compile ejemplo.py".

5. Ejecutar el programa: Para ejecutar el programa, se puede abrir una terminal en el directorio donde se encuentra el archivo ejemplo.py y escribir el siguiente comando:

python ejemplo.py

6. Enlazar el programa: En Python, no es necesario enlazar el programa de forma explícita, ya que Python es un lenguaje interpretado.

En resumen, para editar un programa en Python, se puede usar cualquier editor de texto; para compilarlo, se puede verificar la sintaxis del programa usando el comando "python -m py_compile archivo.py"; para ejecutarlo, se debe usar el comando "python archivo.py".

Bibliografía y Webgrafía

- Joyanes Aguilar, L. (2012). Fundamentos de programación: Lenguaje, estructura y datos de los programas. McGraw Hill.
- González Duque, R. (2020). Introducción a la programación con Python: Algoritmos y lógica de programación para principiantes. Alfaomega.
- Gaddis, T. (2012). Starting Out with Programming Logic and Design (3rd ed.). Pearson.
- González Duque, F. (2019). Aprendiendo a programar con Python 3

Alto nivel: Python es un lenguaje de alto nivel, lo que significa que se enfoca en abstracciones de más alto nivel en lugar de detalles de bajo nivel.

Amplia biblioteca estándar: Python cuenta con una amplia biblioteca estándar que incluye módulos y funciones para realizar tareas comunes como manejo de archivos, conexión de redes, procesamiento de texto, entre otras.

Asignación: la operación de asignar un valor a una variable. En Python, se utiliza el operador de asignación "=", que asigna el valor de la expresión del lado derecho a la variable del lado izquierdo.

Bibliotecas: colecciones de módulos que proporcionan funcionalidades adicionales al lenguaje de programación y permiten realizar tareas complejas de manera más eficiente.

Bloque de código: conjunto de instrucciones agrupadas y ejecutadas como una sola unidad.

Booleanos (bool): valores de Verdadero o Falso (True o False), que se utilizan en declaraciones condicionales.

Break: Palabra clave que permite interrumpir una iteración y salir del ciclo.

Breakpoints: puntos de interrupción en el código que detienen la ejecución del programa en un punto específico para que se puedan examinar los valores de las variables y analizar el flujo del programa.

C#: es un lenguaje de programación desarrollado por Microsoft en la década de 2000. Fue diseñado para la programación de aplicaciones en la plataforma .NET.

C: es un lenguaje de programación de propósito general desarrollado en la década de 1970. Es uno de los lenguajes más populares en la actualidad.

Cadenas de caracteres (str): secuencia de caracteres, como texto o palabras, encerradas entre comillas simples o dobles. Ejemplo: "Hola, mundo".

Caracteres de escape: combinaciones de caracteres especiales que se utilizan para representar caracteres no imprimibles o para controlar la forma en que se muestra la información en la pantalla.

Ciclo: Estructura de control que permite la iteración de un conjunto de instrucciones.
COBOL: es un lenguaje de programación diseñado para aplicaciones empresariales y administrativas. Fue desarrollado en la década de 1950.

Código abierto: es un término utilizado para describir un software cuyo código fuente está disponible públicamente y puede ser utilizado, modificado y distribuido libremente.

Código fuente: es el conjunto de instrucciones escritas en un lenguaje de programación que deben ser traducidas a lenguaje de máquina para que la computadora las entienda.

Compilación: proceso de convertir el código fuente en un lenguaje de bajo nivel para que pueda ser ejecutado por la computadora.

Compilador: es un programa que traduce el código fuente escrito en un lenguaje de programación a lenguaje de máquina para que la computadora lo pueda ejecutar.

Complejos (complex): números complejos, en la forma $a + bj$, donde a y b son números reales y j es la unidad imaginaria.

Comunidad activa: Python tiene una comunidad de desarrolladores activa y comprometida, lo que significa que hay una gran cantidad de recursos en línea, foros de discusión, librerías y herramientas disponibles para los desarrolladores.

Condición: expresión que se evalúa como verdadera o falsa, y se utiliza para controlar el flujo de ejecución de un programa.

Conjuntos (set): colección desordenada de elementos únicos, encerrados entre llaves $\{\}$. Ejemplo: $\{1, 2, 3\}$.

Constante: un valor fijo que no cambia durante la ejecución del programa. Las constantes se utilizan para almacenar valores que se utilizan con frecuencia en el programa y que no deben modificarse.

Continue: Palabra clave que permite interrumpir una iteración y continuar con la siguiente.

Conversión de tipos: la operación de convertir un valor de un tipo de datos a otro. En Python, se utilizan funciones incorporadas como `int()`, `float()`, `str()`, etc. para realizar conversiones de tipos.

Depuración: proceso de identificar, analizar y corregir errores en un programa para que funcione correctamente.

Diccionarios (dict): colección de pares clave-valor, separados por comas y encerrados entre llaves {}. Ejemplo: {"nombre": "Juan", "edad": 30}.

Enteros (int): números enteros, por ejemplo, 5.

Entrada de datos: proceso mediante el cual el usuario proporciona datos al programa a través del teclado o de un archivo externo.

Enumerate: Función que permite recorrer una secuencia y asignar a cada elemento un número de índice.

Estructura de control: instrucciones que controlan el flujo de ejecución de un programa.

Excepciones: eventos inesperados que ocurren durante la ejecución del programa, como errores de sintaxis, errores de tiempo de ejecución y errores de lógica, que pueden ser capturados y manejados para evitar que el programa se bloquee o termine abruptamente.

Expresión: una combinación de valores, variables y operadores que se evalúa para producir un valor. Las expresiones pueden incluir operaciones aritméticas, operaciones de comparación, llamadas a funciones y más.

Fácil de aprender: Python es considerado un lenguaje de programación fácil de aprender debido a su sintaxis clara y legible.

Flotantes (float): números decimales, por ejemplo, 3.14.

For: Ciclo que se utiliza para recorrer una secuencia de elementos (lista, tupla, cadena, etc.) y realizar una acción sobre cada uno de ellos.

Formateo de cadenas: proceso mediante el cual se crea una cadena de salida a partir de una o más variables, utilizando diferentes métodos de formateo para controlar el formato y la precisión de los valores de las variables.

Fortran: es el primer lenguaje de programación de alto nivel, diseñado para la computación científica y de ingeniería. Fue creado en la década de 1950.

Función input(): función incorporada de Python que permite al usuario ingresar datos a través del teclado.

Función `print()`: función incorporada de Python que permite mostrar información en la pantalla.

IDE: acrónimo de Integrated Development Environment, que es un entorno de desarrollo integrado que combina herramientas de programación, depuración y compilación en un solo software.

Indentación: nivel de sangría o espacio en blanco al inicio de una línea de código para indicar la pertenencia a un bloque de código.

Inspección de variables: proceso de examinar los valores de las variables durante la ejecución del programa para identificar errores y optimizar el rendimiento.

Interpretado: Python es un lenguaje interpretado, lo que significa que el código fuente se ejecuta línea por línea en lugar de compilar todo el código antes de su ejecución.

Iteración: Proceso mediante el cual se repite un conjunto de instrucciones un número determinado de veces o hasta que se cumpla una condición específica.

Java: es un lenguaje de programación de propósito general desarrollado en la década de 1990. Es uno de los lenguajes más utilizados en la actualidad, especialmente en la programación de aplicaciones empresariales y móviles.

Lenguaje de máquina: es el lenguaje que entiende directamente una computadora. Está compuesto por una serie de instrucciones en código binario.

Lenguaje de programación: es un lenguaje artificial utilizado para comunicarse con una computadora y darle instrucciones para que realice una tarea específica.

Licencia de software: es un conjunto de términos y condiciones que definen cómo se puede utilizar, modificar y distribuir un software. Las licencias pueden ser propietarias o de código abierto.

Lisp: es un lenguaje de programación utilizado principalmente en inteligencia artificial y procesamiento de lenguaje natural. Fue creado en la década de 1950.

Listas (list): secuencia de elementos, que pueden ser de cualquier tipo de datos, separados por comas y encerrados entre corchetes `[]`. Ejemplo: `[1, "Hola", 3.14]`.

Módulos: archivos que contienen funciones y clases que se pueden importar en otros programas para reutilizar el código y ahorrar tiempo de desarrollo.

Multiplataforma: Python es un lenguaje multiplataforma, lo que significa que se puede utilizar en diferentes sistemas operativos como Windows, macOS y Linux.

Operaciones aritméticas: operaciones matemáticas que se realizan en valores numéricos. En Python, los operadores aritméticos incluyen suma (+), resta (-), multiplicación (*), división (/), división entera (//), módulo (%), y potencia (**).

Operaciones de comparación: operaciones que comparan dos valores y devuelven un valor booleano (True o False) que indica si la comparación es verdadera o falsa. Los operadores de comparación en Python incluyen igual (==), distinto (!=), mayor que (>), menor que (<), mayor o igual que (>=), y menor o igual que (<=).

Operador ternario: un operador que toma tres operandos y se utiliza como una forma abreviada de una declaración if-else.

Operador ternario: un operador que toma tres operandos y se utiliza como una forma abreviada de una declaración if-else.

Operador: un símbolo que indica una operación que se debe realizar en uno o más valores. Los operadores en Python incluyen operadores aritméticos, operadores de comparación, operadores lógicos, operadores de asignación y más.

Operadores de asignación: operadores utilizados para asignar un valor a una variable.

Operadores de comparación: símbolos utilizados para comparar valores y evaluar condiciones, como == (igual a), != (distinto de), > (mayor que), < (menor que), >= (mayor o igual que), <= (menor o igual que).

Operadores de identidad: operadores utilizados para comparar la identidad de dos objetos.

Operadores de pertenencia: operadores utilizados para comprobar si un valor se encuentra dentro de una secuencia o colección.

Operadores lógicos: símbolos utilizados para combinar o invertir valores lógicos, como and (y), or (o), not (no).

Orientado a objetos: Python es un lenguaje orientado a objetos, lo que significa que se basa en el concepto de clases y objetos, y permite la creación de programas modulares y reutilizables.

Python: es un lenguaje de programación interpretado de alto nivel diseñado para ser fácil de leer y escribir. Fue creado a finales de la década de 1980.

Range: Función que permite generar una secuencia de números enteros dentro de un rango determinado.

Ruby: es un lenguaje de programación interpretado de alto nivel utilizado en aplicaciones web y móviles. Fue creado a mediados de la década de 1990.

Salida de datos: proceso mediante el cual el programa muestra información al usuario a través de la pantalla o de un archivo externo.

Secuenciación: ordenamiento lógico de instrucciones en un programa.

Sentencia elif: estructura de control condicional que permite evaluar múltiples condiciones y ejecutar distintos bloques de código según el resultado de dichas evaluaciones.

Sentencia else: estructura de control condicional que permite ejecutar un bloque de código si la condición de una sentencia if es falsa.

Sentencia if: estructura de control condicional que permite ejecutar un bloque de código si una condición es verdadera.

Swift: es un lenguaje de programación desarrollado por Apple en la década de 2010 para la programación de aplicaciones en iOS, macOS y otros sistemas operativos de la empresa.

Tipado dinámico: una característica de Python en la que el tipo de una variable se determina en tiempo de ejecución en función del valor que se le asigna. Esto significa que no es necesario especificar el tipo de una variable antes de usarla.

Traceback: un mensaje de error que muestra la secuencia de llamadas de funciones y la línea de código donde se produjo una excepción, que ayuda a identificar la causa del error y corregirlo.

Tuplas (tuple): secuencia de elementos, que pueden ser de cualquier tipo de datos, separados por comas y encerrados entre paréntesis (). Ejemplo: (1, "Hola", 3.14).

Variable: un espacio reservado en la memoria que almacena un valor o una referencia a un objeto. El valor almacenado puede cambiar durante la ejecución del programa.

Virtual environment: entorno de desarrollo aislado que se puede configurar para trabajar con una versión específica de Python y sus dependencias, sin afectar a otras aplicaciones o proyectos en el sistema.

While: Ciclo que se utiliza para repetir un conjunto de instrucciones mientras se cumpla una condición determinada.